

```
In [1]: !pip install requests
!pip install pandas
!pip install numpy
```

```
Requirement already satisfied: requests in c:\users\michael\anaconda3a\lib\site-packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (2025.1.31)
Requirement already satisfied: pandas in c:\users\michael\anaconda3a\lib\site-packages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: six>=1.5 in c:\users\michael\anaconda3a\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Requirement already satisfied: numpy in c:\users\michael\anaconda3a\lib\site-packages (1.26.4)
```

```
In [2]: # Requests allows us to make HTTP requests which we will use to get data from an API
import requests
# Pandas is a software library written for the Python programming language for data manipulation and analysis
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays
import numpy as np
# Datetime is a library that allows us to represent dates
import datetime
```

```
In [3]: # Setting this option will print all columns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)
```

```
In [4]: # Takes the dataset and uses the rocket column to call the API and append the data to the BoosterVersion list
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x))
            BoosterVersion.append(response['name'])
```

```
In [5]: # Takes the dataset and uses the launchpad column to call the API and append the data to the LaunchSite list
def getLaunchSite(data):
    for x in data['launchpad']:
        if x:
```

```

response = requests.get("https://api.spacexdata.com/v4/launchpads/"+str(Longitude.append(response['longitude']))
Latitude.append(response['latitude']))
LaunchSite.append(response['name'])

```

```

In [6]: # Takes the dataset and uses the payloads column to call the API and append the
def getPayloadData(data):
    for load in data['payloads']:
        if load:
            response = requests.get("https://api.spacexdata.com/v4/payloads/"+load)
            PayloadMass.append(response['mass_kg'])
            Orbit.append(response['orbit'])

```

```

In [7]: # Takes the dataset and uses the cores column to call the API and append the data
def getCoreData(data):
    for core in data['cores']:
        if core['core'] != None:
            response = requests.get("https://api.spacexdata.com/v4/cores/"+core['core'])
            Block.append(response['block'])
            ReusedCount.append(response['reuse_count'])
            Serial.append(response['serial'])
        else:
            Block.append(None)
            ReusedCount.append(None)
            Serial.append(None)
        Outcome.append(str(core['landing_success'])+' '+str(core['landing_type']))
        Flights.append(core['flight'])
        GridFins.append(core['gridfins'])
        Reused.append(core['reused'])
        Legs.append(core['legs'])
        LandingPad.append(core['landpad'])

```

```

In [8]: spacex_url="https://api.spacexdata.com/v4/launches/past"

```

```

In [9]: response = requests.get(spacex_url)

```

```

In [10]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud'

```

```

In [11]: response=requests.get(static_json_url)

```

```

In [12]: response.status_code

```

```

Out[12]: 200

```

```

In [13]: import requests
import pandas as pd

# URL of the static JSON file
static_json_url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud'

# Step 1: Fetch the static JSON file from the URL
response = requests.get(static_json_url)

# Step 2: Check if the request was successful

```

```
if response.status_code == 200:
    print("✅ Data fetched successfully!")
    data_json = response.json() # Convert the response to JSON
else:
    print(f"❌ Failed to fetch data. Status code: {response.status_code}")

# Step 3: Normalize the JSON and convert to a DataFrame
data = pd.json_normalize(data_json)

# Step 4: Display DataFrame info
print("\n📊 Data shape:", data.shape) # Displays the number of rows and column
print("\n📋 Columns:")
print(data.columns.tolist()) # Lists all the columns

# Step 5: Display first few rows
data.head() # Shows a preview of the first few rows
```

✅ Data fetched successfully!

📊 Data shape: (107, 42)

📋 Columns:

```
['static_fire_date_utc', 'static_fire_date_unix', 'tbd', 'net', 'window', 'rocket', 'success', 'details', 'crew', 'ships', 'capsules', 'payloads', 'launchpad', 'auto_update', 'failures', 'flight_number', 'name', 'date_utc', 'date_unix', 'date_local', 'date_precision', 'upcoming', 'cores', 'id', 'fairings.reused', 'fairings.recovery_attempt', 'fairings.recovered', 'fairings.ships', 'links.patch.small', 'links.patch.large', 'links.reddit.campaign', 'links.reddit.launch', 'links.reddit.media', 'links.reddit.recovery', 'links.flickr.small', 'links.flickr.original', 'links.presskit', 'links.webcast', 'links.youtube_id', 'links.article', 'links.wikipedia', 'fairings']
```

Out[13]:

	static_fire_date_utc	static_fire_date_unix	tbd	net	window
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	False	0.0 5e9d0d95eda699
1	None	NaN	False	False	0.0 5e9d0d95eda699
2	None	NaN	False	False	0.0 5e9d0d95eda699
3	2008-09-20T00:00:00.000Z	1.221869e+09	False	False	0.0 5e9d0d95eda699

static_fire_date_utc static_fire_date_unix tbd net window

4 None NaN False False 0.0 5e9d0d95eda699

```
In [23]: # Lets take a subset of our dataframe keeping only the features we want and the
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_

# We will remove rows with multiple cores because those are falcon rockets with
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single v
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extractin
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the Launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

```
In [15]: # Lets take a subset of our dataframe keeping only the features we want and the
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_

# We will remove rows with multiple cores because those are falcon rockets with
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single v
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extractin
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the Launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

```
In [24]: #Global variables
BoosterVersion = []
```

```
PayloadMass = []  
Orbit = []  
LaunchSite = []  
Outcome = []  
Flights = []  
GridFins = []  
Reused = []  
Legs = []  
LandingPad = []  
Block = []  
ReusedCount = []  
Serial = []  
Longitude = []  
Latitude = []
```

```
In [25]: BoosterVersion
```

```
Out[25]: []
```

```
In [26]: # Call getBoosterVersion  
getBoosterVersion(data)
```

```
In [27]: BoosterVersion[0:5]
```

```
Out[27]: []
```

```
In [28]: # Call getLaunchSite  
getLaunchSite(data)
```

```
In [29]: # Call getPayloadData  
getPayloadData(data)
```

```
In [30]: # Call getCoreData  
getCoreData(data)
```

```
In [32]: launch_dict = {'FlightNumber': list(data['flight_number']),  
                        'Date': list(data['date']),  
                        'BoosterVersion':BoosterVersion,  
                        'PayloadMass':PayloadMass,  
                        'Orbit':Orbit,  
                        'LaunchSite':LaunchSite,  
                        'Outcome':Outcome,  
                        'Flights':Flights,  
                        'GridFins':GridFins,  
                        'Reused':Reused,  
                        'Legs':Legs,  
                        'LandingPad':LandingPad,  
                        'Block':Block,  
                        'ReusedCount':ReusedCount,  
                        'Serial':Serial,  
                        'Longitude': Longitude,  
                        'Latitude': Latitude}
```

```
In [33]: !pip3 install beautifulsoup4
```

```
!pip3 install requests
```

```
Requirement already satisfied: beautifulsoup4 in c:\users\michael\anaconda3a\lib\site-packages (4.12.3)
Requirement already satisfied: soupsieve>1.2 in c:\users\michael\anaconda3a\lib\site-packages (from beautifulsoup4) (2.5)
Requirement already satisfied: requests in c:\users\michael\anaconda3a\lib\site-packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\michael\anaconda3a\lib\site-packages (from requests) (2025.1.31)
```

```
In [34]: import sys
```

```
import requests
from bs4 import BeautifulSoup
import re
import unicodedata
import pandas as pd
```

```
In [35]: def date_time(table_cells):
```

```
    """
    This function returns the data and time from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
```

```
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]
```

```
def booster_version(table_cells):
```

```
    """
    This function returns the booster version from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
```

```
    out=''.join([booster_version for i,booster_version in enumerate(table_cells.strings)])
    return out
```

```
def landing_status(table_cells):
```

```
    """
    This function returns the landing status from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
```

```
    out=[i for i in table_cells.strings][0]
    return out
```

```
def get_mass(table_cells):
```

```
    mass=unicodedata.normalize("NFKD", table_cells.text).strip()
    if mass:
        mass.find("kg")
        new_mass=mass[0:mass.find("kg")+2]
    else:
        new_mass=0
```

```

        return new_mass

def extract_column_from_header(row):
    """
    This function returns the landing status from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    if (row.br):
        row.br.extract()
    if row.a:
        row.a.extract()
    if row.sup:
        row.sup.extract()

    column_name = ' '.join(row.contents)

    # Filter the digit and empty names
    if not(column_name.strip().isdigit()):
        column_name = column_name.strip()
    return column_name

```

In [36]: `static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Fa`

```

In [37]: import requests

# The static URL of the Falcon 9 and Falcon Heavy Launches Wikipedia page
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Fa

# Perform GET request
response = requests.get(static_url)

# Check the status
print("Status Code:", response.status_code)

```

Status Code: 200

```

In [38]: from bs4 import BeautifulSoup

# Create a BeautifulSoup object from the response content
soup = BeautifulSoup(response.text, "html.parser")

# Optional: Check the first 500 characters of the parsed HTML to verify it's working
print(soup.prettify()[:500]) # Prints the first 500 characters in a nicely formatted way

<!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled vector-feature-language-in-main-page-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector"

```

```

In [39]: # Print the title of the page
print("Page Title:", soup.title.string)

```


Page Title: List of Falcon 9 and Falcon Heavy launches - Wikipedia

```
In [40]: # Use find_all to find all tables on the page
html_tables = soup.find_all("table")

# Check how many tables were found
print(f"Number of tables found: {len(html_tables)}")
```

Number of tables found: 25

```
In [41]: # Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)
```

```
<table class="wikitable plainrowheaders collapsible" style="width: 100%;">
<tbody><tr>
<th scope="col">Flight No.
</th>
<th scope="col">Date and<br/>time (<a href="/wiki/Coordinated_Universal_Time" ti
tle="Coordinated Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="Lis
t of Falcon 9 first-stage boosters">Version,<br/>Booster</a> <sup class="referen
ce" id="cite_ref-booster_11-0"><a href="#cite_note-booster-11"><span class="cit
e-bracket">[</span>b<span class="cite-bracket">]</span></a></sup>
</th>
<th scope="col">Launch site
</th>
<th scope="col">Payload<sup class="reference" id="cite_ref-Dragon_12-0"><a href
="#cite_note-Dragon-12"><span class="cite-bracket">[</span>c<span class="cite-br
acket">]</span></a></sup>
</th>
<th scope="col">Payload mass
</th>
<th scope="col">Orbit
</th>
<th scope="col">Customer
</th>
<th scope="col">Launch<br/>outcome
</th>
<th scope="col"><a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon
9 first-stage landing tests">Booster<br/>landing</a>
</th></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">1
</th>
<td>4 June 2010,<br/>18:45
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-0"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B0003.1<sup class="reference" id="cite_ref-block_numbers_14-0"><a href="#cite
_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-br
acket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/Dragon_Spacecraft_Qualification_Unit" title="Dragon Spacecraf
t Qualification Unit">Dragon Spacecraft Qualification Unit</a>
</td>
<td>
</td>
<td>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/SpaceX" title="SpaceX">SpaceX</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success
```

```
</td>
<td class="table-failure" style="background: #FFC7C7; color:black; vertical-align: middle; text-align: center;">Failure<sup class="reference" id="cite_ref-ns20110930_15-0"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-16"><a href="#cite_note-16"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup><br><small>(parachute)</small>
</td></tr>
<tr>
<td colspan="9">First flight of Falcon 9 v1.0.<sup class="reference" id="cite_ref-sfn20100604_17-0"><a href="#cite_note-sfn20100604-17"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup> Used a boilerplate version of Dragon capsule which was not designed to separate from the second stage.<small><a href="#First_flight_of_Falcon_9">more details below</a></small> Attempted to recover the first stage by parachuting it into the ocean, but it burned up on reentry, before the parachutes even deployed.<sup class="reference" id="cite_ref-parachute_18-0"><a href="#cite_note-parachute-18"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">2
</th>
<td>8 December 2010,<br>15:43<sup class="reference" id="cite_ref-spaceflightnow_Clark_Launch_Report_19-0"><a href="#cite_note-spaceflightnow_Clark_Launch_Report-19"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="reference" id="cite_ref-MuskMay2012_13-1"><a href="#cite_note-MuskMay2012-13"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup><br><b>B0004.1</b><sup class="reference" id="cite_ref-block_numbers_14-1"><a href="#cite_note-block_numbers-14"><span class="cite-bracket">[</span><span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Space Force Station">CCAFS</a>,<br><a href="/wiki/Cape_Canaveral_Space_Launch_Complex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_Dragon" title="SpaceX Dragon">Dragon</a> <a class="mw-redirect" href="/wiki/COTS_Demo_Flight_1" title="COTS Demo Flight 1">demo flight C1</a><br>(Dragon C101)
</td>
<td>
</td>
<td>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/wiki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><style data-mw-deduplicate="TemplateStyles:r1126788409">.mw-parser-output .plainlist ol,.mw-parser-output .plainlist ul{line-height:inherit;list-style:none;margin:0;padding:0}.mw-parser-output .plainlist ol li,.mw-parser-output .plainlist ul li{margin-bottom:0}</style><div class="plainlist">
<ul><li><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Orbital_Transportation_Services" title="Commercial Orbital Transportation Services">COTS</a>)</li>
<li><a href="/wiki/National_Reconnaissance_Office" title="National Reconnaissance Office">NRO</a></li></ul>
</td>
```

```
</div>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-align: middle; text-align: center;">Success<sup class="reference" id="cite_ref-ns20110930_15-1"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</span>9<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-failure" style="background: #FFC7C7; color:black; vertical-align: middle; text-align: center;">Failure<sup class="reference" id="cite_ref-ns20110930_15-2"><a href="#cite_note-ns20110930-15"><span class="cite-bracket">[</span>9<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-20"><a href="#cite_note-20"><span class="cite-bracket">[</span>14<span class="cite-bracket">]</span></a></sup><br/><small>(parachute)</small>
</td></tr>
<tr>
<td colspan="9">Maiden flight of <a class="mw-redirect" href="/wiki/Dragon_capsule" title="Dragon capsule">Dragon capsule</a>, consisting of over 3 hours of testing thruster maneuvering and reentry.<sup class="reference" id="cite_ref-spaceflightnow_Clark_unleashing_Dragon_21-0"><a href="#cite_note-spaceflightnow_Clark_unleashing_Dragon-21"><span class="cite-bracket">[</span>15<span class="cite-bracket">]</span></a></sup> Attempted to recover the first stage by parachuting it into the ocean, but it disintegrated upon reentry, before the parachutes were deployed.<sup class="reference" id="cite_ref-parachute_18-1"><a href="#cite_note-parachute-18"><span class="cite-bracket">[</span>12<span class="cite-bracket">]</span></a></sup> <small>( <a href="#COTS_demo_missions">more details below</a>)</small> It also included two <a href="/wiki/CubeSat" title="CubeSat">CubeSats</a>, <sup class="reference" id="cite_ref-NRO_Taps_Boeing_for_Next_Batch_of_CubeSats_22-0"><a href="#cite_note-NRO_Taps_Boeing_for_Next_Batch_of_CubeSats-22"><span class="cite-bracket">[</span>16<span class="cite-bracket">]</span></a></sup> and a wheel of <a href="/wiki/Brou%C3%A8re" title="Brouère">Brouère</a> cheese.
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">3
</th>
<td>22 May 2012,<br/>07:44<sup class="reference" id="cite_ref-BBC_new_era_23-0"><a href="#cite_note-BBC_new_era-23"><span class="cite-bracket">[</span>17<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="reference" id="cite_ref-MuskMay2012_13-2"><a href="#cite_note-MuskMay2012-13"><span class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><br/>B0005.1<sup class="reference" id="cite_ref-block_numbers_14-2"><a href="#cite_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Space Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Complex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_Dragon" title="SpaceX Dragon">Dragon</a> <a class="mw-redirect" href="/wiki/Dragon_C2%2B" title="Dragon C2+">demo flight C2+</a><sup class="reference" id="cite_ref-C2_24-0"><a href="#cite_note-C2-24"><span class="cite-bracket">[</span>18<span class="cite-bracket">]</span></a></sup><br/>(Dragon C102)
</td>
<td>525 kg (1,157 lb)<sup class="reference" id="cite_ref-25"><a href="#cite_note
```

```
e-25"><span class="cite-bracket">[</span>19<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/wiki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Orbital_Transportation_Services" title="Commercial Orbital Transportation Services">COTS</a>)
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-align: middle; text-align: center;">Success<sup class="reference" id="cite_ref-26"><a href="#cite_note-26"><span class="cite-bracket">[</span>20<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-align: middle; white-space: nowrap; text-align: center;">No attempt
</td></tr>
<tr>
<td colspan="9">Dragon spacecraft demonstrated a series of tests before it was allowed to approach the <a href="/wiki/International_Space_Station" title="International Space Station">International Space Station</a>. Two days later, it became the first commercial spacecraft to board the ISS.<sup class="reference" id="cite_ref-BBC_new_era_23-1"><a href="#cite_note-BBC_new_era-23"><span class="cite-bracket">[</span>17<span class="cite-bracket">]</span></a></sup> <small><a href="#COTS_demo_missions">more details below</a></small>
</td></tr>
<tr>
<th rowspan="3" scope="row" style="text-align:center;">4
</th>
<td rowspan="2">8 October 2012,<br/>00:35<sup class="reference" id="cite_ref-SFN_LLog_27-0"><a href="#cite_note-SFN_LLog-27"><span class="cite-bracket">[</span>21<span class="cite-bracket">]</span></a></sup>
</td>
<td rowspan="2"><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="reference" id="cite_ref-MuskMay2012_13-3"><a href="#cite_note-MuskMay2012-13"><span class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><br/>B0006.1<sup class="reference" id="cite_ref-block_numbers_14-3"><a href="#cite_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-bracket">]</span></a></sup>
</td>
<td rowspan="2"><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Space Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Complex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_CRS-1" title="SpaceX CRS-1">SpaceX CRS-1</a><sup class="reference" id="cite_ref-sxManifest20120925_28-0"><a href="#cite_note-sxManifest20120925-28"><span class="cite-bracket">[</span>22<span class="cite-bracket">]</span></a></sup><br/>(Dragon C103)
</td>
<td>4,700 kg (10,400 lb)
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a href="/wiki/International_Space_Station" title="International Space Station">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Resupp
```

```
ly_Services" title="Commercial Resupply Services">CRS</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-align: middle; text-align: center;">Success
</td>
<td rowspan="2" style="background:#ececce; text-align:center;"><span class="nowrap">No attempt</span>
</td></tr>
<tr>
<td><a href="/wiki/Orbcomm_(satellite)" title="Orbcomm (satellite)">Orbcomm-OG2
</a><sup class="reference" id="cite_ref-Orbcomm_29-0"><a href="#cite_note-Orbcomm-29"><span class="cite-bracket">[</span>23<span class="cite-bracket">]</span></a></sup>
</td>
<td>172 kg (379 lb)<sup class="reference" id="cite_ref-gunter-og2_30-0"><a href="#cite_note-gunter-og2-30"><span class="cite-bracket">[</span>24<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/Orbcomm" title="Orbcomm">Orbcomm</a>
</td>
<td class="table-partial" style="background: #FFB; color:black; vertical-align: middle; text-align: center;">Partial failure<sup class="reference" id="cite_ref-nyt-20121030_31-0"><a href="#cite_note-nyt-20121030-31"><span class="cite-bracket">[</span>25<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<td colspan="9">CRS-1 was successful, but the <a href="/wiki/Secondary_payload" title="Secondary payload">secondary payload</a> was inserted into an abnormally low orbit and subsequently lost. This was due to one of the nine <a href="/wiki/SpaceX_Merlin" title="SpaceX Merlin">Merlin engines</a> shutting down during the launch, and NASA declining a second reignition, as per <a href="/wiki/International_Space_Station" title="International Space Station">ISS</a> visiting vehicle safety rules, the primary payload owner is contractually allowed to decline a second reignition. NASA stated that this was because SpaceX could not guarantee a high enough likelihood of the second stage completing the second burn successfully which was required to avoid any risk of secondary payload's collision with the ISS.<sup class="reference" id="cite_ref-OrbcommTotalLoss_32-0"><a href="#cite_note-OrbcommTotalLoss-32"><span class="cite-bracket">[</span>26<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-sn20121011_33-0"><a href="#cite_note-sn20121011-33"><span class="cite-bracket">[</span>27<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-34"><a href="#cite_note-34"><span class="cite-bracket">[</span>28<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">5
</th>
<td>1 March 2013,<br/>15:10
</td>
<td><a href="/wiki/Falcon_9_v1.0" title="Falcon 9 v1.0">F9 v1.0</a><sup class="reference" id="cite_ref-MuskMay2012_13-4"><a href="#cite_note-MuskMay2012-13"><span class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><br/>B0007.1<sup class="reference" id="cite_ref-block_numbers_14-4"><a href="#cite_note-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-bracket">]</span></a></sup>
```

```

    <td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Spa
ce Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Comp
lex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
<td><a href="/wiki/SpaceX_CRS-2" title="SpaceX CRS-2">SpaceX CRS-2</a><sup class
="reference" id="cite_ref-sxManifest20120925_28-1"><a href="#cite_note-sxManifes
t20120925-28"><span class="cite-bracket">[</span>22<span class="cite-bracke
t">]</span></a></sup><br/>(Dragon C104)
</td>
<td>4,877 kg (10,752 lb)
</td>
<td><a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a> (<a class="m
w-redirect" href="/wiki/ISS" title="ISS">ISS</a>)
</td>
<td><a href="/wiki/NASA" title="NASA">NASA</a> (<a href="/wiki/Commercial_Resupp
ly_Services" title="Commercial Resupply Services">CRS</a>)
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-alig
n: middle; text-align: center;">Success
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-alig
n: middle; white-space: nowrap; text-align: center;">No attempt
</td></tr>
<tr>
<td colspan="9">Last launch of the original Falcon 9 v1.0 <a href="/wiki/Launch_
vehicle" title="Launch vehicle">launch vehicle</a>, first use of the unpressuriz
ed trunk section of Dragon.<sup class="reference" id="cite_ref-sxf9_20110321_3
5-0"><a href="#cite_note-sxf9_20110321-35"><span class="cite-bracket">[</span>29
<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">6
</th>
<td>29 September 2013,<br/>16:00<sup class="reference" id="cite_ref-pa20130930_3
6-0"><a href="#cite_note-pa20130930-36"><span class="cite-bracket">[</span>30<sp
an class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.1" title="Falcon 9 v1.1">F9 v1.1</a><sup class="r
eference" id="cite_ref-MuskMay2012_13-5"><a href="#cite_note-MuskMay2012-13"><sp
an class="cite-bracket">[</span>7<span class="cite-bracket">]</span></a></sup><b
r/>B1003<sup class="reference" id="cite_ref-block_numbers_14-5"><a href="#cite_n
ote-block_numbers-14"><span class="cite-bracket">[</span>8<span class="cite-brac
ket">]</span></a></sup>
</td>
<td><a class="mw-redirect" href="/wiki/Vandenberg_Air_Force_Base" title="Vandenb
erg Air Force Base">VAFB</a>,<br/><a href="/wiki/Vandenberg_Space_Launch_Complex
_4" title="Vandenberg Space Launch Complex 4">SLC-4E</a>
</td>
<td><a href="/wiki/CASSIOPE" title="CASSIOPE">CASSIOPE</a><sup class="reference"
id="cite_ref-sxManifest20120925_28-2"><a href="#cite_note-sxManifest20120925-2
8"><span class="cite-bracket">[</span>22<span class="cite-bracket">]</span></a>
</sup><sup class="reference" id="cite_ref-CASSIOPE_MDA_37-0"><a href="#cite_not
e-CASSIOPE_MDA-37"><span class="cite-bracket">[</span>31<span class="cite-bracke
t">]</span></a></sup>

```

```
</td>
<td>500 kg (1,100 lb)
</td>
<td><a href="/wiki/Polar_orbit" title="Polar orbit">Polar orbit</a> <a href="/wiki/Low_Earth_orbit" title="Low Earth orbit">LEO</a>
</td>
<td><a href="/wiki/Maxar_Technologies" title="Maxar Technologies">MDA</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-align: middle; text-align: center;">Success<sup class="reference" id="cite_ref-pa20130930_36-1"><a href="#cite_note-pa20130930-36"><span class="cite-bracket">[</span>30<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-no2" style="background: #FFE3E3; color: black; vertical-align: middle; text-align: center;">Uncontrolled<br/><small>(ocean)</small><sup class="reference" id="cite_ref-ocean_landing_38-0"><a href="#cite_note-ocean_landing-38"><span class="cite-bracket">[</span>d<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<td colspan="9">First commercial mission with a private customer, first launch from Vandenberg, and demonstration flight of Falcon 9 v1.1 with an improved 13-tonne to LEO capacity.<sup class="reference" id="cite_ref-sxf9_20110321_35-1"><a href="#cite_note-sxf9_20110321-35"><span class="cite-bracket">[</span>29<span class="cite-bracket">]</span></a></sup> After separation from the second stage carrying Canadian commercial and scientific satellites, the first stage booster performed a controlled reentry,<sup class="reference" id="cite_ref-39"><a href="#cite_note-39"><span class="cite-bracket">[</span>32<span class="cite-bracket">]</span></a></sup> and an <a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon 9 first-stage landing tests">ocean touchdown test</a> for the first time. This provided good test data, even though the booster started rolling as it neared the ocean, leading to the shutdown of the central engine as the roll depleted it of fuel, resulting in a hard impact with the ocean.<sup class="reference" id="cite_ref-pa20130930_36-2"><a href="#cite_note-pa20130930-36"><span class="cite-bracket">[</span>30<span class="cite-bracket">]</span></a></sup> This was the first known attempt of a rocket engine being lit to perform a supersonic retro propulsion, and allowed SpaceX to enter a public-private partnership with <a href="/wiki/NASA" title="NASA">NASA</a> and its Mars entry, descent, and landing technologies research projects.<sup class="reference" id="cite_ref-40"><a href="#cite_note-40"><span class="cite-bracket">[</span>33<span class="cite-bracket">]</span></a></sup> <small>(<a href="#Maiden_flight_of_v1.1">more details below</a>)</small>
</td></tr>
<tr>
<th rowspan="2" scope="row" style="text-align:center;">7
</th>
<td>3 December 2013,<br/>22:41<sup class="reference" id="cite_ref-sfn_wwls20130624_41-0"><a href="#cite_note-sfn_wwls20130624-41"><span class="cite-bracket">[</span>34<span class="cite-bracket">]</span></a></sup>
</td>
<td><a href="/wiki/Falcon_9_v1.1" title="Falcon 9 v1.1">F9 v1.1</a><br/>B1004
</td>
<td><a href="/wiki/Cape_Canaveral_Space_Force_Station" title="Cape Canaveral Space Force Station">CCAFS</a>,<br/><a href="/wiki/Cape_Canaveral_Space_Launch_Complex_40" title="Cape Canaveral Space Launch Complex 40">SLC-40</a>
</td>
```



```

<td><a href="/wiki/SES-8" title="SES-8">SES-8</a><sup class="reference" id="cite_ref-sxManifest20120925_28-3"><a href="#cite_note-sxManifest20120925-28"><span class="cite-bracket">[</span>22<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-spx-pr_42-0"><a href="#cite_note-spx-pr-42"><span class="cite-bracket">[</span>35<span class="cite-bracket">]</span></a></sup><sup class="reference" id="cite_ref-aw20110323_43-0"><a href="#cite_note-aw20110323-43"><span class="cite-bracket">[</span>36<span class="cite-bracket">]</span></a></sup></td>
<td>3,170 kg (6,990 lb)
</td>
<td><a href="/wiki/Geostationary_transfer_orbit" title="Geostationary transfer orbit">GTO</a>
</td>
<td><a class="mw-redirect" href="/wiki/SES_S.A." title="SES S.A.">SES</a>
</td>
<td class="table-success" style="background: #9EFF9E; color:black; vertical-align: middle; text-align: center;">Success<sup class="reference" id="cite_ref-SNMissionStatus7_44-0"><a href="#cite_note-SNMissionStatus7-44"><span class="cite-bracket">[</span>37<span class="cite-bracket">]</span></a></sup>
</td>
<td class="table-noAttempt" style="background: #EEE; color:black; vertical-align: middle; white-space: nowrap; text-align: center;">No attempt<br><sup class="reference" id="cite_ref-sf10120131203_45-0"><a href="#cite_note-sf10120131203-45"><span class="cite-bracket">[</span>38<span class="cite-bracket">]</span></a></sup>
</td></tr>
<tr>
<td colspan="9">First <a href="/wiki/Geostationary_transfer_orbit" title="Geostationary transfer orbit">Geostationary transfer orbit</a> (GTO) launch for Falcon 9,<sup class="reference" id="cite_ref-spx-pr_42-1"><a href="#cite_note-spx-pr-42"><span class="cite-bracket">[</span>35<span class="cite-bracket">]</span></a></sup> and first successful reignition of the second stage.<sup class="reference" id="cite_ref-46"><a href="#cite_note-46"><span class="cite-bracket">[</span>39<span class="cite-bracket">]</span></a></sup> SES-8 was inserted into a <a href="/wiki/Geostationary_transfer_orbit" title="Geostationary transfer orbit">Super-Synchronous Transfer Orbit</a> of 79,341 km (49,300 mi) in apogee with an <a href="/wiki/Orbital_inclination" title="Orbital inclination">inclination</a> of 2 0.55° to the <a href="/wiki/Equator" title="Equator">equator</a>.
</td></tr></tbody></table>

```

```

In [42]: # Initialize an empty list to hold the column names
column_names = []

# Iterate through all <th> elements in the first_launch_table
for th in first_launch_table.find_all("th"):
    # Apply the provided extract_column_from_header() function to extract the column name
    column_name = extract_column_from_header(th)

    # Append the non-empty column name to the list
    if column_name and len(column_name) > 0:
        column_names.append(column_name)

# Print the column names
print("Column Names:", column_names)

```

Column Names: ['Flight No.', 'Date and time ()', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome']

```
In [43]: print(column_names)
```

['Flight No.', 'Date and time ()', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome']

```
In [44]: # Initialize an empty dictionary with column names as keys
launch_dict = dict.fromkeys(column_names)
```

```
# Remove the irrelevant column
```

```
del launch_dict['Date and time ( )']
```

```
# Initialize each column with an empty list
```

```
launch_dict['Flight No.'] = []
```

```
launch_dict['Launch site'] = []
```

```
launch_dict['Payload'] = []
```

```
launch_dict['Payload mass'] = []
```

```
launch_dict['Orbit'] = []
```

```
launch_dict['Customer'] = []
```

```
launch_dict['Launch outcome'] = []
```

```
# Add the new columns
```

```
launch_dict['Version Booster'] = []
```

```
launch_dict['Booster landing'] = []
```

```
launch_dict['Date'] = []
```

```
launch_dict['Time'] = []
```

```
# Print the dictionary to check its structure
```

```
print(launch_dict)
```

```
{'Flight No.': [], 'Launch site': [], 'Payload': [], 'Payload mass': [], 'Orbit': [], 'Customer': [], 'Launch outcome': [], 'Version Booster': [], 'Booster landing': [], 'Date': [], 'Time': []}
```

```
In [45]: # Create the DataFrame from the dictionary
```

```
df = pd.DataFrame({key: pd.Series(value) for key, value in launch_dict.items()})
```

```
# Display the first few rows of the DataFrame to verify
```

```
print(df.head())
```

Empty DataFrame

Columns: [Flight No., Launch site, Payload, Payload mass, Orbit, Customer, Launch outcome, Version Booster, Booster landing, Date, Time]

Index: []

```
In [46]: # Export the DataFrame to a CSV file
```

```
df.to_csv('spacex_web_scraped.csv', index=False)
```

```
# Confirm the CSV has been saved
```

```
print("CSV file 'spacex_web_scraped.csv' has been successfully saved.")
```

CSV file 'spacex_web_scraped.csv' has been successfully saved.

```
In [47]: # Filter the dataframe to only include Falcon 9 Launches
```

```
falcon_9_df = df[df['Payload'].str.contains('Falcon 9', na=False)]
```

```
# Display the filtered dataframe
print(falcon_9_df.head())
```

Empty DataFrame

Columns: [Flight No., Launch site, Payload, Payload mass, Orbit, Customer, Launch outcome, Version Booster, Booster landing, Date, Time]
Index: []

```
In [48]: # Replace NaN values in 'Payload mass' with the mean of that column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

Series([], Name: Payload mass, dtype: object)

C:\Users\Michael\AppData\Local\Temp\ipykernel_14288\1234062768.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

```
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)
```

```
In [49]: # Remove 'kg' and commas, then convert to numeric values
df['Payload mass'] = df['Payload mass'].str.replace(' kg', '', regex=False) # Remove 'kg'
df['Payload mass'] = df['Payload mass'].str.replace(',', '', regex=False) # Remove commas

# Convert the column to numeric values (this will convert invalid parsing to NaN)
df['Payload mass'] = pd.to_numeric(df['Payload mass'], errors='coerce')

# Replace NaN values with the mean of the column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

Series([], Name: Payload mass, dtype: int64)

C:\Users\Michael\AppData\Local\Temp\ipykernel_14288\4205711417.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

```
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)
```

```
In [50]: # Remove 'kg' and commas, then convert to numeric values
df['Payload mass'] = df['Payload mass'].str.replace(' kg', '', regex=False) # Remove 'kg'
df['Payload mass'] = df['Payload mass'].str.replace(',', '', regex=False) # Remove commas

# Convert the column to numeric values (this will convert invalid parsing to NaN)
df['Payload mass'] = pd.to_numeric(df['Payload mass'], errors='coerce')

# Replace NaN values with the mean of the column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[50], line 2
      1 # Remove 'kg' and commas, then convert to numeric values
----> 2 df['Payload mass'] = df['Payload mass'].str.replace(' kg', '', regex=False)
se) # Remove ' kg'
      3 df['Payload mass'] = df['Payload mass'].str.replace(',', '', regex=False)
e) # Remove commas
      5 # Convert the column to numeric values (this will convert invalid parsing
to NaN)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:6299, in NDFrame.__get_attr__(self, name)
    6292 if (
    6293     name not in self._internal_names_set
    6294     and name not in self._metadata
    6295     and name not in self._accessors
    6296     and self._info_axis._can_hold_identifiers_and_holds_name(name)
    6297 ):
    6298     return self[name]
-> 6299 return object.__getattribute__(self, name)

File ~\anaconda3\Lib\site-packages\pandas\core\accessor.py:224, in CachedAccessor.__get__(self, obj, cls)
    221 if obj is None:
    222     # we're accessing the attribute of the class, i.e., Dataset.geo
    223     return self._accessor
--> 224 accessor_obj = self._accessor(obj)
    225 # Replace the property with the accessor object. Inspired by:
    226 # https://www.pydanny.com/cached-property.html
    227 # We need to use object.__setattr__ because we overwrite __setattr__ on
    228 # NDFrame
    229 object.__setattr__(obj, self._name, accessor_obj)

File ~\anaconda3\Lib\site-packages\pandas\core\strings\accessor.py:191, in StringMethods.__init__(self, data)
    188 def __init__(self, data) -> None:
    189     from pandas.core.arrays.string_ import StringDtype
--> 191     self._inferred_dtype = self._validate(data)
    192     self._is_categorical = isinstance(data.dtype, CategoricalDtype)
    193     self._is_string = isinstance(data.dtype, StringDtype)

File ~\anaconda3\Lib\site-packages\pandas\core\strings\accessor.py:245, in StringMethods._validate(self, data)
    242 inferred_dtype = lib.infer_dtype(values, skipna=True)
    244 if inferred_dtype not in allowed_types:
--> 245     raise AttributeError("Can only use .str accessor with string values!")
    246 return inferred_dtype

```

AttributeError: Can only use .str accessor with string values!

```

In [51]: # Filter the dataframe to only include Falcon 9 launches
         falcon_9_df = df[df['Payload'].str.contains('Falcon 9', na=False)]

         # Display the filtered dataframe

```

```
print(falcon_9_df.head())
```

Empty DataFrame

Columns: [Flight No., Launch site, Payload, Payload mass, Orbit, Customer, Launch outcome, Version Booster, Booster landing, Date, Time]

Index: []

```
In [52]: # Replace NaN values in 'Payload mass' with the mean of that column
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)

# Display the updated DataFrame to verify
print(df['Payload mass'].head())
```

Series([], Name: Payload mass, dtype: int64)

C:\Users\Michael\AppData\Local\Temp\ipykernel_14288\1234062768.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

```
df['Payload mass'].fillna(df['Payload mass'].mean(), inplace=True)
```

```
In [53]: import requests
import pandas as pd

# Step 1: Send GET request to the SpaceX API
url = "https://api.spacexdata.com/v4/launches" # SpaceX API URL for Launches
response = requests.get(url)

# Step 2: Convert the response JSON into a DataFrame
launch_data = response.json()
df = pd.json_normalize(launch_data)

# Step 3: Filter for Falcon 9 launches (assuming the vehicle info is in a key called 'vehicles')
# This assumes the column with the rocket type name is 'rocket' and Falcon 9 launches are in the 'rocket' column
falcon_9_df = df[df['rocket'].str.contains('Falcon 9', case=False, na=False)]

# Step 4: Remove Falcon 1 launches by filtering them out (assuming 'Falcon 1' is in the 'rocket' column)
falcon_9_df = falcon_9_df[~falcon_9_df['rocket'].str.contains('Falcon 1', case=False)]

# Step 5: Count the number of Falcon 9 launches
falcon_9_launch_count = len(falcon_9_df)

# Print the count of Falcon 9 launches after removing Falcon 1 launches
print(f"Number of Falcon 9 launches after removing Falcon 1 launches: {falcon_9_launch_count}")
```

Number of Falcon 9 launches after removing Falcon 1 launches: 0

In []: